

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РЕСПУБЛИКИ КАЗАХСТАН

КАЗАХСТАНСКО-НЕМЕЦКИЙ УНИВЕРСИТЕТ

Факультет инжиниринга и информационных технологий
Образовательная программа: 6B06129 – Computer Science

Дисциплина: Алгоритмы, структуры данных и программирование

Практическая работа №4
Тема:Стеки, очереди и связанные списки.

Выполнил: студент 1-го курса
Слюсарь Артур
Проверила: Жантлеуова А.К.

Алматы, 2026

Содержание:

1. 20 Valid Parentheses (стек).....	3
2. 155 Min Stack (стек).....	4
3. 150 Evaluate Reverse Polish Notation (стек).....	5
4. 232 Implement Queue using Stacks (очередь).....	6
5. 622 Design Circular Queue (очередь).....	7
6. 707 Design Linked List (связанный список).....	8

Задание №1 20 Valid Parentheses (стек)

Идея решения:

Мы идем по строке слева направо. Если видим открывающую скобку - мы отправляем её в стек. Это наше «обещание» закрыть её в будущем. Если видим закрывающую скобку, мы проверяем вершину стека. Если стек пуст или скобка на вершине стека не совпадает по типу с закрывающей - строка выдаст False. Если скобки образуют правильную пару, мы удаляем открывающую скобку из стека и идем дальше. В самом конце, после прохода по всей строке, мы проверяем стек. Если он пуст, значит, все скобки успешно закрылись и строка выдаст True. Если в стеке что-то осталось, строка выдаст False.

На рисунке 1 представлен скриншот принятого решения:

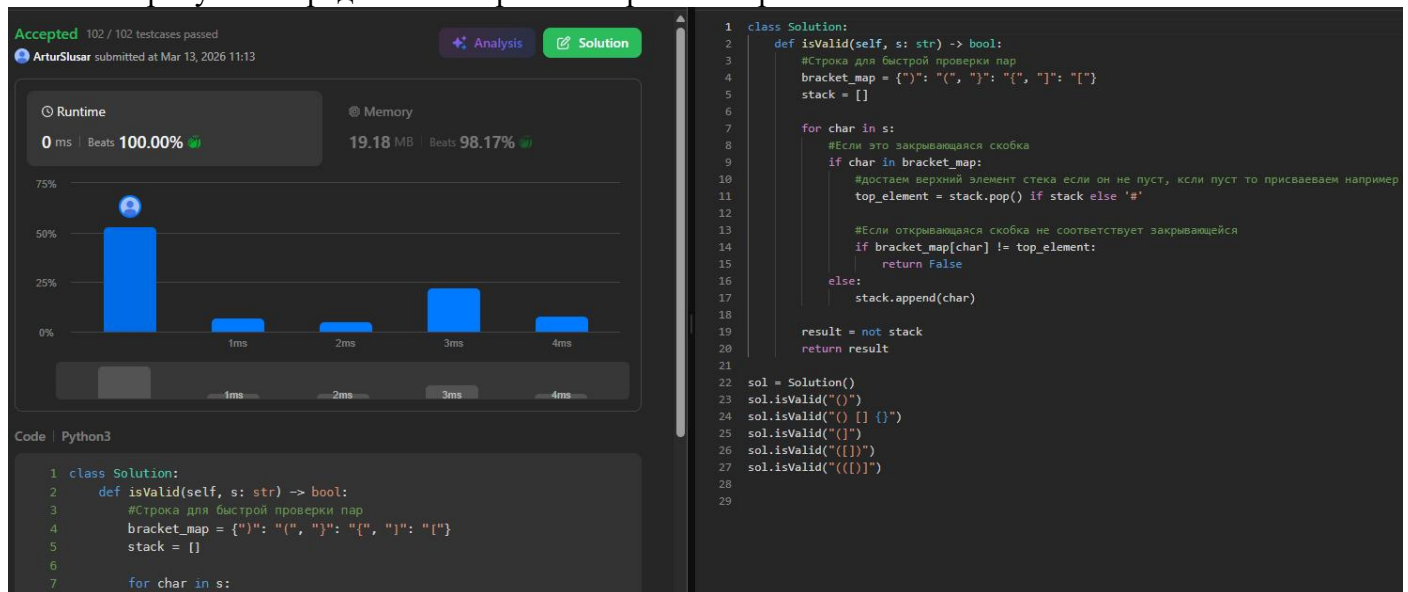


Рисунок №1

Анализ решения:

1. Временная сложность

Мы проходим по каждому символу строки ровно один раз. Все операции внутри цикла - выполняются за константное время $O(1)$. Итоговое время линейно зависит от длины строки.

2. Сложность по памяти:

В худшем случае нам придется положить в стек все символы строки. Таким образом, размер стека может вырасти пропорционально длине входной строки. Словарь `bracket_map` всегда занимает $O(1)$ памяти, так как содержит фиксированное количество элементов, и на асимптотику не влияет.

Задание №2

155 Min Stack (стек)

Идея решения:

Стек должен помнить историю минимумов. Для этого мы храним в стеке не одиночные числа, а пары: (само число, минимум на данный момент).

На рисунке 2 представлен скриншот принятого решения:

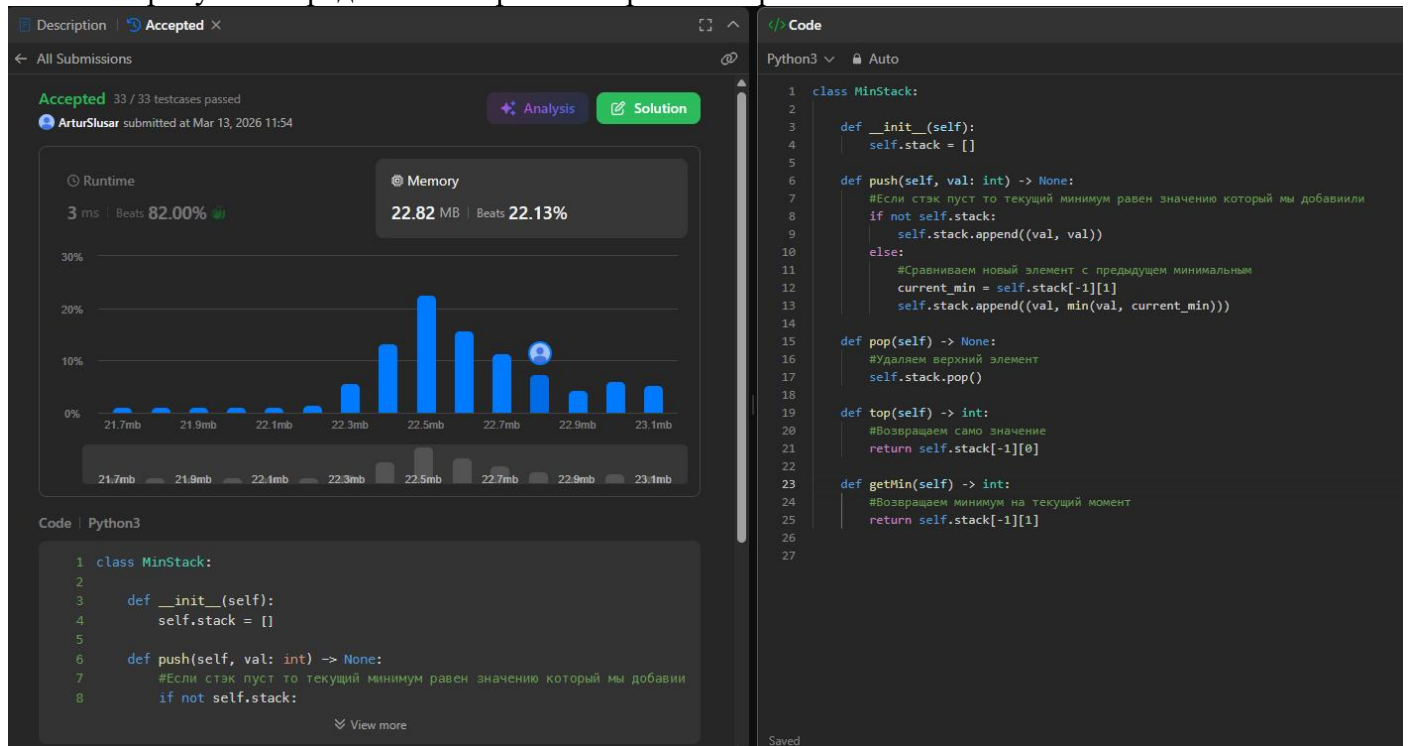


Рисунок №2

Анализ решения:

1. Временная сложность:

$O(1)$ для абсолютно всех методов (push, pop, top, getMin). Так как мы нигде не используем циклы

2. Сложность по памяти:

Для каждого добавленного числа мы вынуждены хранить еще одно дополнительное значение текущего минимума. Стек занимает в два раза больше памяти, чем обычный массив, но это все равно линейная зависимость $O(n)$.

Задание №3

150 Evaluate Reverse Polish Notation (стек)

Идея решения:

Мы проходим по массиву слева направо и если встречается число, мы переводим его в целый тип и кладем в стек, а если математический знак - достаем из стека два последних добавленных числа. Мы применяем этот оператор к нашим числам и кладем результат обратно в стек. По условию оно должно усекается к нулю, поэтому вместо стандартного целочисленного деления `//`, мы используем конструкцию `int(a / b)`. Когда мы пройдем весь массив, все промежуточные операции будут выполнены, и в стеке останется ровно одно число

На рисунке 3 представлен скриншот принятого решения:

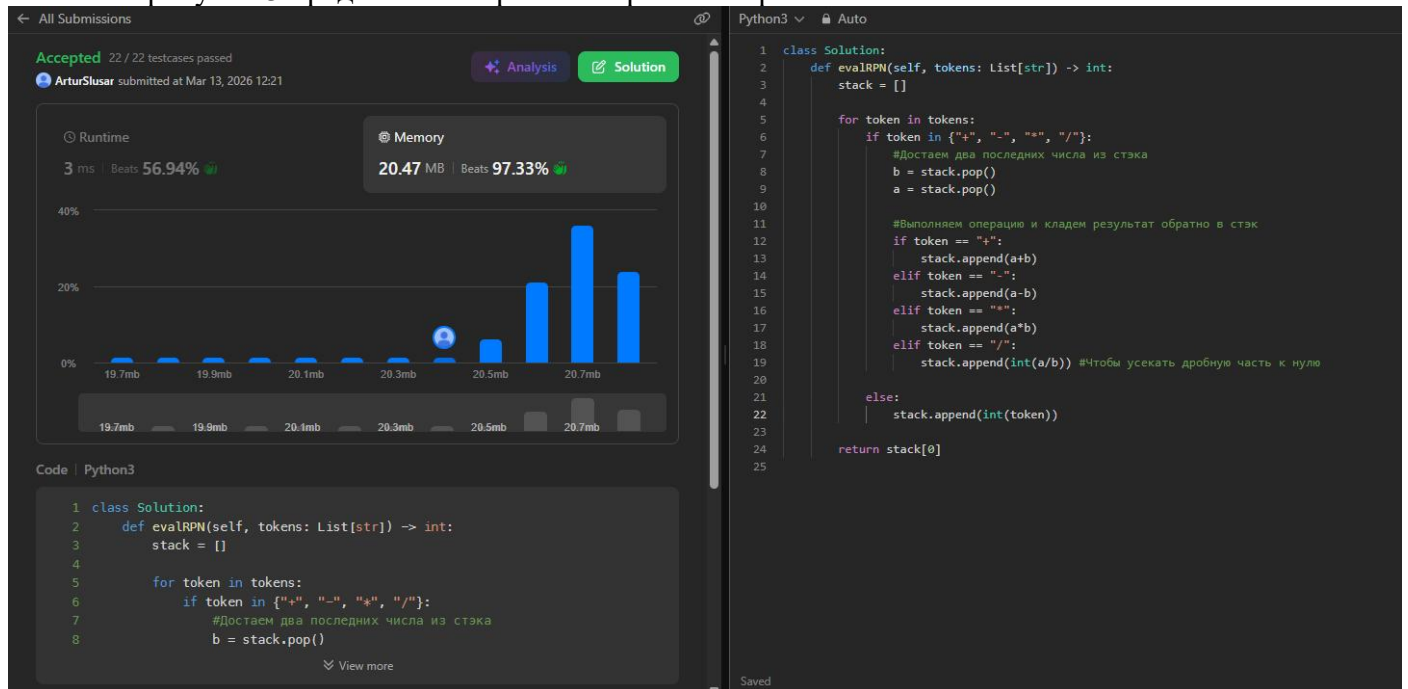


Рисунок №3

Анализ решения:

1. Временная сложность:

Мы проходим по списку ровно один раз. Все математические операции и взаимодействия со стеком в Python выполняются за константное время $O(1)$.

2. Сложность по памяти:

В худшем сценарии (например, когда в начале выражения идет множество чисел подряд, а все знаки стоят в самом конце) стек будет вынужден хранить в себе почти все числа из массива одновременно, прежде чем начнутся вычисления. Поэтому потребление памяти напрямую зависит от длины входных данных $O(n)$.

Задание 4. 232 Implement Queue using Stacks (очередь)

Идея решения:

Мы используем Стек. Проходим по массиву слева направо. Числа переводим в `int` и добавляем в стек. Операторы извлекают два последних числа из стека. Вычисляем результат и кладем его обратно. В конце алгоритма в стеке останется ровно одно число - это и есть ответ.

На рисунке 4 представлен скриншот принятого решения:

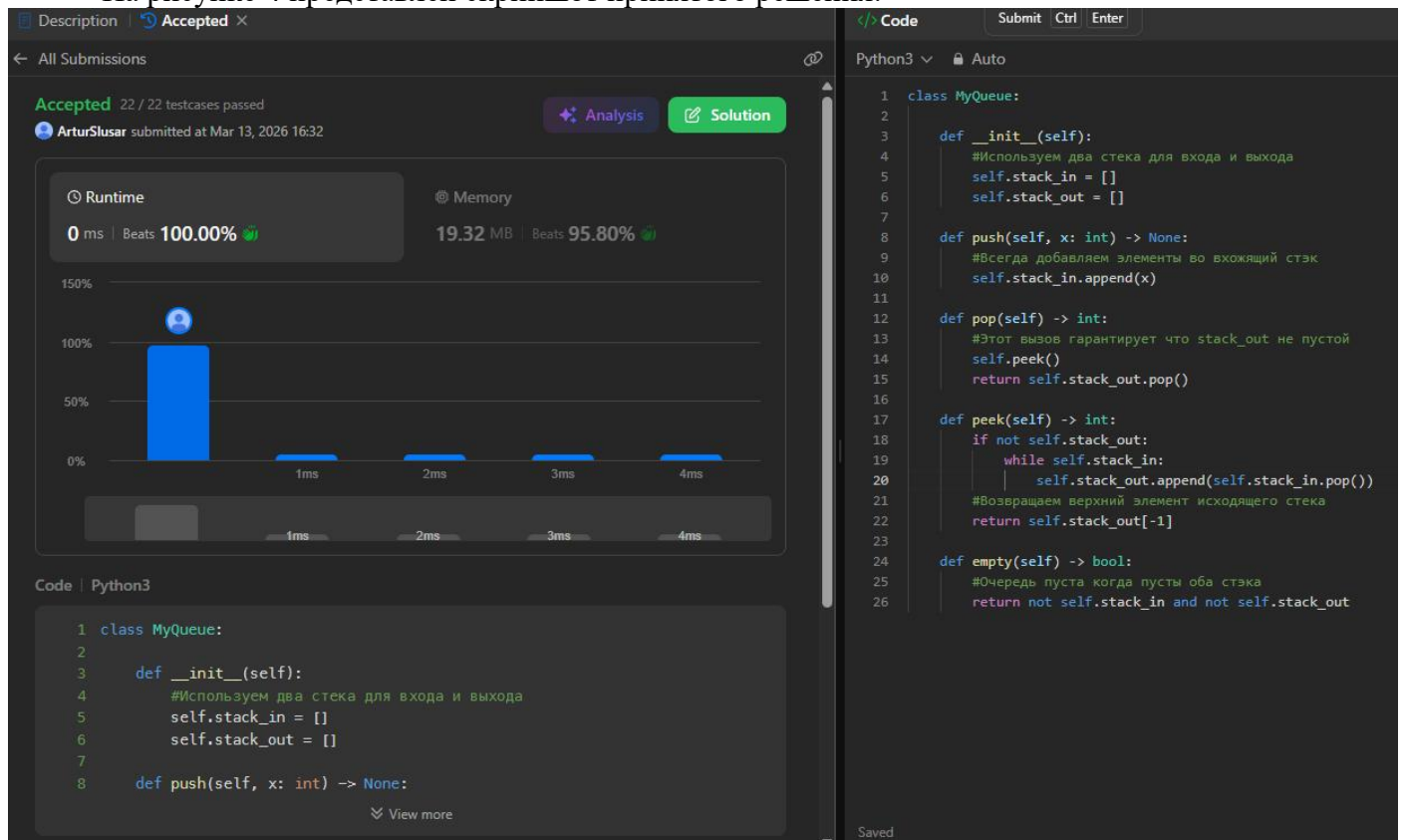


Рисунок №4

Анализ решения:

1. Временная сложность:

Мы обходим массив всего один раз, а операции со стеком (`push/pop`) и вычисления занимают $O(1)$.

2. Сложность по памяти:

В худшем случае $O(n)$ нам придется хранить в стеке почти все числа из массива одновременно.

Задание 5. 622 Design Circular Queue (очередь)

Идея решения:

Мы используем обычный массив фиксированного размера и два указателя, начало (head) и конец (tail). При добавлении элемента мы двигаем tail вперед, а при удалении - двигаем head. Если любой из указателей доходит до конца массива, мы просто сбрасываем его в ноль через if. За счет этого массив работает как бесконечное кольцо, и освободившиеся ячейки используются по кругу.

На рисунке 5 представлен скриншот принятого решения:

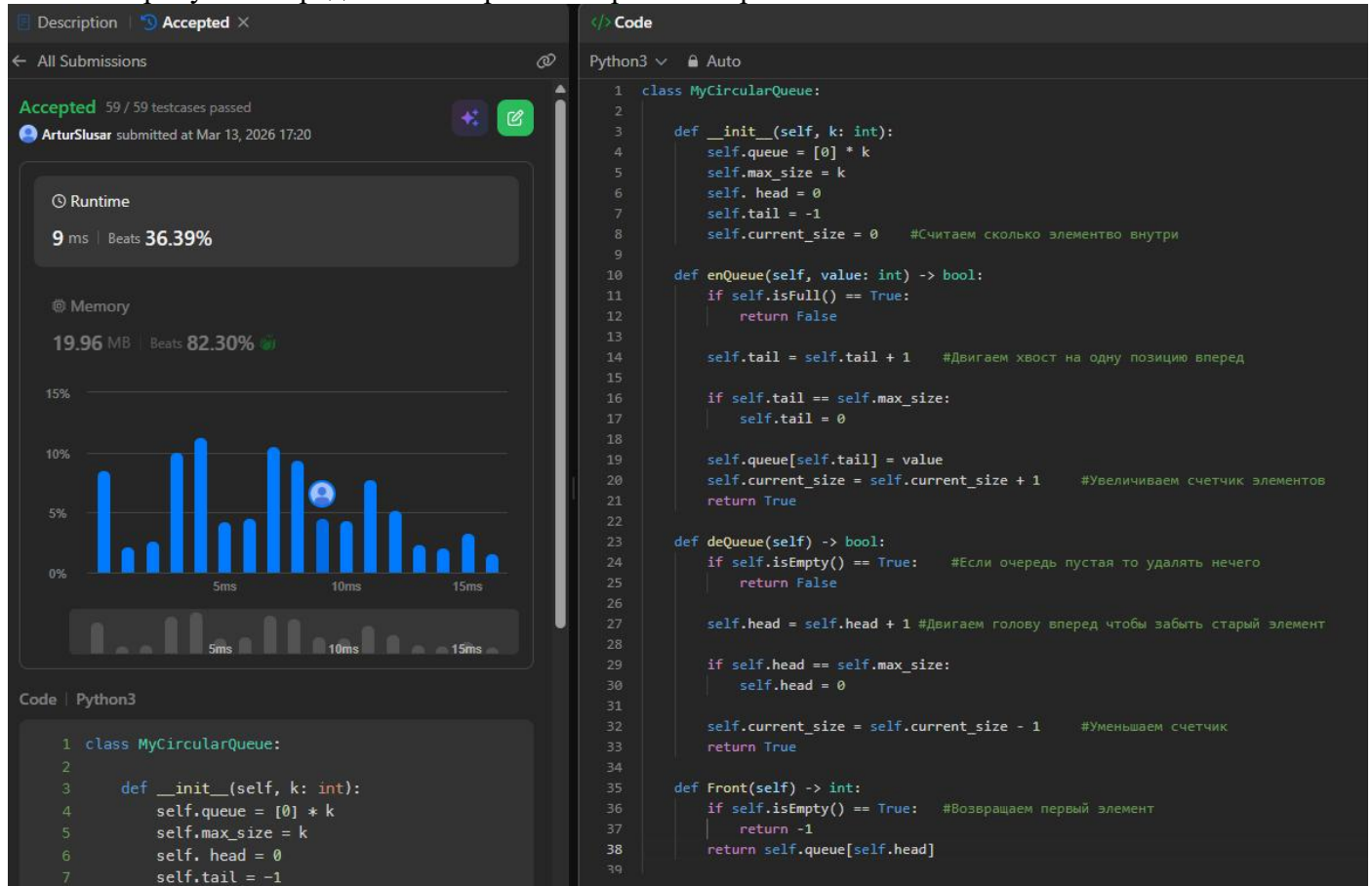


Рисунок №5

Анализ решения:

1. Временная сложность:

$O(1)$ для всех операций. Мы не используем циклы. Любое действие - это просто сложение, проверка через if и мгновенное обращение к ячейке массива по индексу.

2. Сложность по памяти:

Память зависит только от входного числа $O(n)$. Мы ровно один раз создаем массив на n элементов при запуске программы. Больше этот массив не растет и не сужается, мы лишь гоняем по нему указатели.

Задание 6. 707 Design Linked List (связанный список)

Идея решения:

Чтобы найти, добавить или удалить элемент по индексу, мы стартуем от головы (head) и шагаем по ссылкам с помощью цикла. Для вставки или удаления мы останавливаемся на узле *перед* нужной позицией и просто перекидываем его указатель next на новый или следующий узел.

На рисунке 6 представлен скриншот принятого решения:

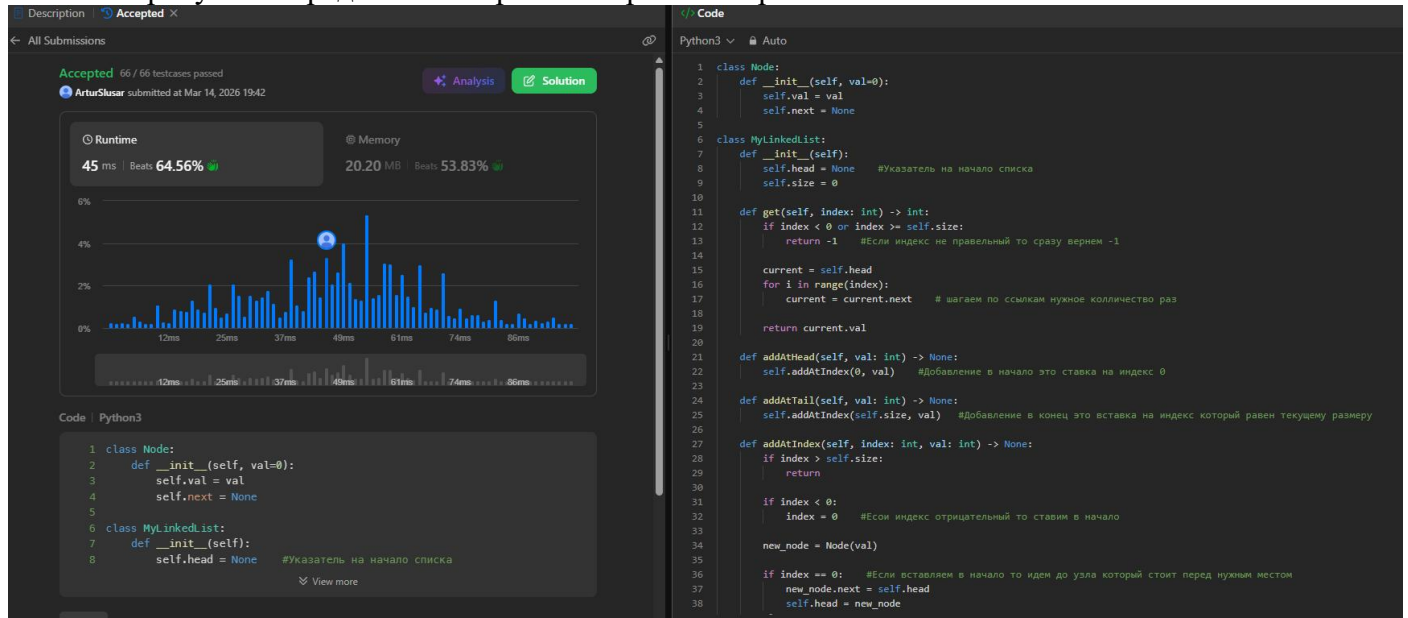


Рисунок №6

Анализ решения:

1. Временная сложность:

$O(1)$ так как добавление в начало (addAtHead), так как мы просто обновляем указатель головы без перебора.

2. Сложность по памяти:

$O(n)$ так как для get, addAtTail, addAtIndex и deleteAtIndex. Нам приходится идти по ссылкам с самого начала списка, чтобы добраться до нужного индекса.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. GitHub // <https://github.com/ArturSlusar/Slusar-A> (был доступен 24.02.2026)